Project - Low Level Design

On

Autonomous Retail Researcher Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Tanish Israni	EN22IT301112
2	Samarth Patidar	EN22IT301087
3	Yash Dashore	EN22CS3011100
4	Ramakant Singh	EN22IT301077
5	Abhishek Gupta	EN22EL301004
6	Sourabh Patel	EN22IT301106

Group Name: Group 10D12

Project Number: AAI-01

Industry Mentor Name: Ashwin sir

University Mentor Name: Ajeet Singh Rajput

Academic Year: 2026

Table of Contents

1. **Introduction :**

The Autonomous Retail Research Agent (v2) is a web-based intelligent system designed to automate the process of generating retail market research reports. The system leverages advanced Generative AI technologies and multi-agent orchestration to transform a simple user query into a comprehensive, structured, and professional research report. By integrating Large Language Models (LLMs) with real-time web search capabilities, the system ensures that the generated insights are both relevant and up-to-date.

In today's fast-paced business environment, conducting retail research manually is time-consuming and requires domain expertise. This system addresses that challenge by enabling users to generate high-quality reports instantly through a simple web interface. The platform not only enhances productivity but also democratizes access to market intelligence by making it available to non-technical users through an intuitive chat-based interface.

1.1. Scope of the document :

This document provides a detailed Low-Level Design (LLD) of the Autonomous Retail Research Agent (v2). It outlines the system architecture, internal components, workflows, and interactions between different modules.

The document specifically covers:

1. Application architecture and layered design
2. Process and information flow across the system
3. Detailed component-level design
4. API structure and communication mechanisms
5. Key design considerations such as scalability, performance, and security

It serves as a technical guide for implementation and future enhancements of the system.

1.2. Intended Audience :

This document is intended for a wide range of stakeholders involved in the development and evaluation of the system. It is primarily designed for software developers and engineers who will implement the system components and integrate various technologies such as AI models, APIs, and frontend frameworks.

Additionally, this document is useful for project evaluators, academic reviewers, and mentors who need to understand the system's design and architecture. A basic understanding of APIs, and artificial intelligence concepts is assumed.

1.3. System overview :

The Autonomous Retail Research Agent (v2) is a web-first application that allows users to generate retail market research reports through a simple prompt. The system uses an agent-based architecture powered by CrewAI, where multiple agents collaborate to perform tasks such as research, analysis, and report generation.

The workflow begins when a user enters a query in the web interface built using Streamlit. The input is processed by the backend, where agents utilize LangChain tools to fetch real-time data via web search APIs. The collected information is then processed using LLMs such as GPT-4o or Gemini to generate a structured report. The final output is displayed in a chat-style interface and can be downloaded by the user.

2. System Design :

The system follows a modular and scalable architecture, where frontend, backend, and AI components are clearly separated. This design ensures flexibility, maintainability, and ease of integration with external services.

2.1. Application Design :

The application is divided into multiple layers to ensure separation of concerns and efficient system management.

Frontend Layer

The frontend is built using Streamlit and provides an interactive web interface for users. It includes:

- A search bar for user input
- A chat-style output display
- Real-time progress updates
- A download button for reports

This layer is responsible for user interaction and presentation.

Backend Layer

The backend is implemented in Python and acts as the core processing unit of the system.

It handles:

- Input validation
- Agent orchestration using CrewAI
- Communication with LLMs and APIs
- Report generation and formatting

The backend ensures smooth coordination between different components.

AI & Services Layer • This layer includes:

- LLMs (GPT-4o / Gemini) for text generation
- Web search tools (Serper API via LangChain)
- Agent logic for task execution

This layer is responsible for intelligence and decision-making within the system.

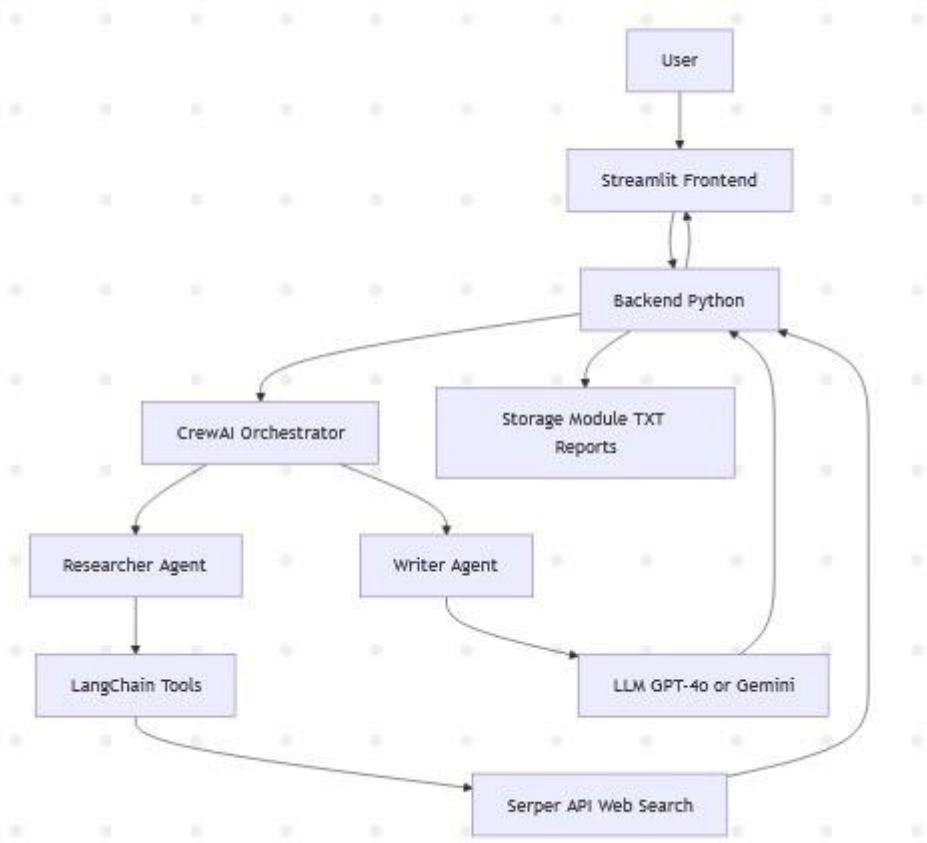


Fig 2.1 : Application Design

2.2. Process Flow :

The system follows a structured yet dynamic process flow, similar to the sequential workflow.Process Flow A

Step-by-Step Flow

1. User enters a research query in the web interface
2. Frontend sends the request to the backend
3. Backend validates the input
4. CrewAI initializes agents

5. Researcher Agent performs web search
6. Writer Agent compiles and structures the report
7. LLM processes and formats the content
8. Final report is generated
9. Output is displayed in chat format
10. Report is saved and made downloadable

This flow ensures efficient and automated report generation.

Activity Diagram :-

2.3. Information Flow :

The information flow in the system ensures smooth data movement between components.

Data Flow

Input Data: User prompt Processing Data:

Search results

Extracted insights

Structured intermediate data Output Data:

Final research report (Markdown + TXT)

Flow Direction

User → Frontend → Backend → Agents → LLM → Backend → Frontend → User

This structured flow ensures minimal latency and efficient processing.

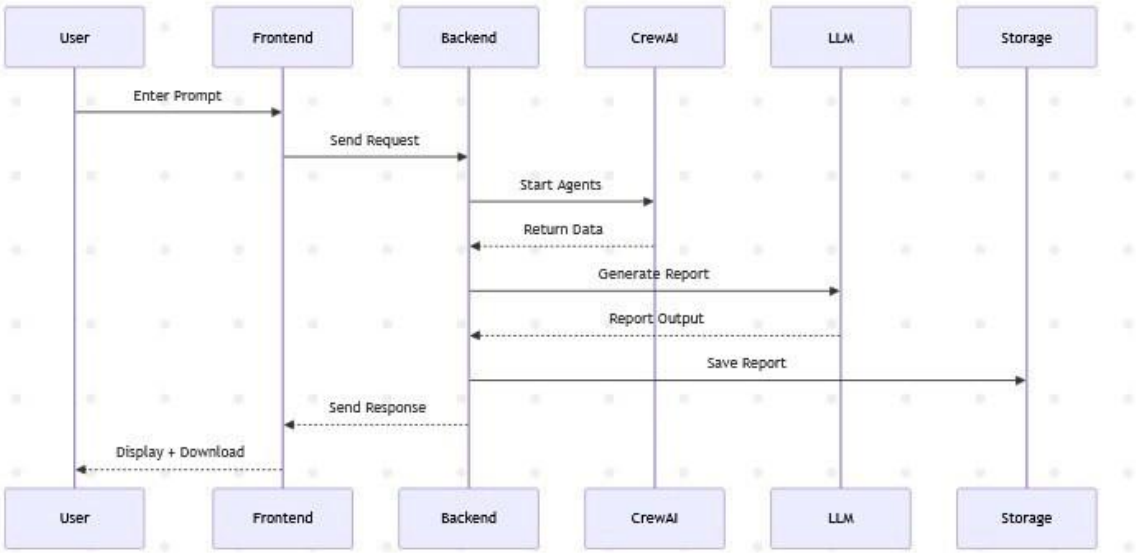


Fig 3.3 : Information Flow

2.4. Components Design :

The system is composed of several interconnected components.

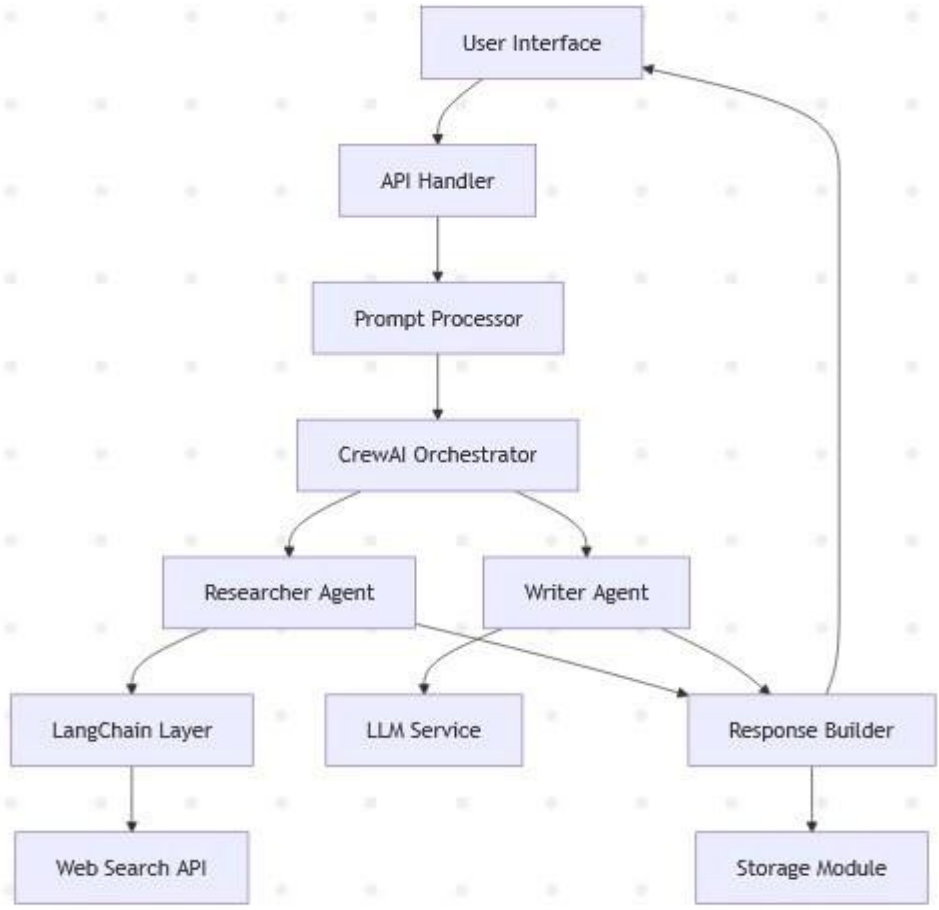
1. User Interface
   - Input field for queries
   - Chat-based output display
   - Download functionality

2. Agent Orchestrator (CrewAI)
   - Manages multiple agents

- Coordinates tasks between agents

3. Researcher Agent
   - Performs web search
   - Collects relevant data
4. Writer Agent
   - Structures and formats the report
   - Generates final output
5. LLM Module
   - Processes natural language
   - Generates insights and summaries
6. Storage Module
   - Saves reports as .txt files
   - Maintains internal repository

Component Design Diagram -



2.5. Key Design Considerations :

Several important factors were considered while designing the system

1. Scalability
   The system is designed to handle multiple user requests simultaneously using modular architecture and cloud deployment.

2. Performance
   Efficient processing is ensured through optimized agent workflows and minimal UI blocking.

3. Modularity
   Each component is independent, allowing easy updates or replacement (e.g., switching LLMs).

4. Usability
   The system provides a clean and intuitive interface requiring no technical expertise.

5. Reliability
   Error handling mechanisms are implemented for API failures, invalid inputs, and timeouts.

6. Security
   API keys are stored securely using environment variables and secrets management.

2.6. API Catalogue :

1. Generate Report API

Method: POST
Endpoint: /generate-report *Request:*

```
{
  "prompt": "Retail market trends in India 2025"
}
```

Processing Steps:

- Validate input
- Trigger CrewAI agents
- Fetch data using search API
- Generate report using LLM

Format output

Response:

```
{
  "status": "success",
  "report": "Generated structured report..."
}
```

2. Download API
Method: GET
Endpoint: /download-report

Response:

Returns downloadable .txt file

3. Error Handling
- Empty input → Validation error
- API failure → Fallback response
- Timeout → Retry or notify user

2.7. Class Diagram :

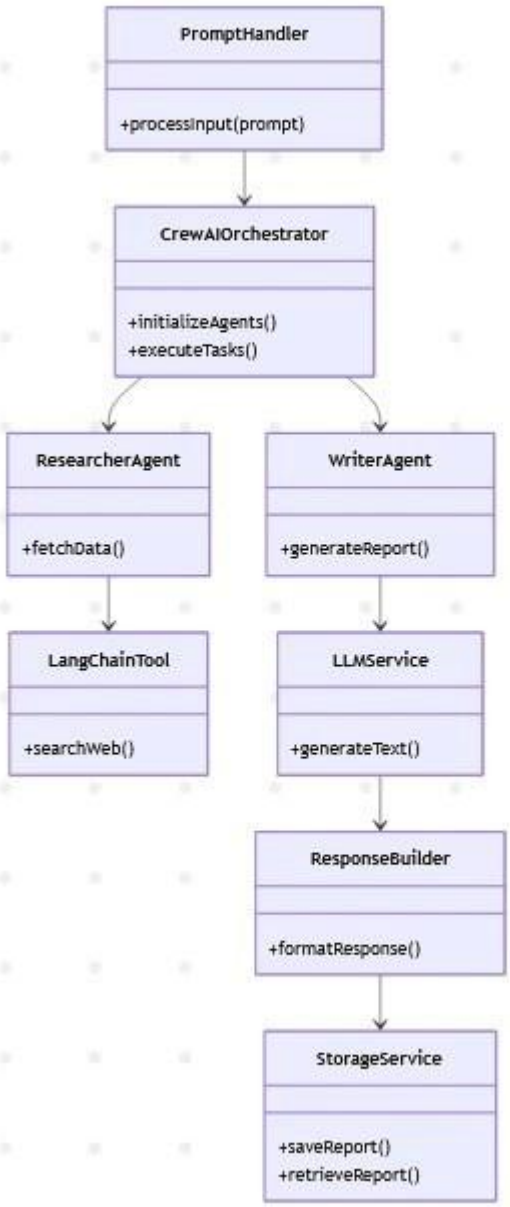


Fig 2.7 : Class Diagram

3. References

1. CrewAI Documentation
2. LangChain Documentation
3. Streamlit Documentation
4. OpenAI & Gemini API Docs
5. Serper API Documentation